Dr. Vishwanath Karad
MIT WORLD PEACE UNIVERSITY | PUNE
TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

# COURSE STRUCTURE

| | | | | |
|---|---|---|---|---|
| **Course Code** | **CET3018B** | | | |
| **Course Category** | **Professional Core** | | | |
| **Course Title** | **Distributed Computing** | | | |
| **Teaching Scheme and Credits Weekly load hrs** | **Lecture** | **Tutorial** | **Laboratory** | **Credits** |
| | **3 hr/wk** | -- | **2hrs/wk** | **3+1=4** |

**Pre-requisites**:

- Computer Networks
- Operating systems

**Course Objectives:**

**1. Knowledge:**

. To study principles of distributed computing.

i. To understand the concept of cloud computing.

**2. Skills:**

. To study and implement various distributed algorithms used in distributed system design for distributed computing.

i. To implement middleware technologies.

**Course Outcomes:**

After completion of this course students will be able to: -

1. Configure and set-up distributed computing environments, by the usage of modern tools.
2. To understand the importance of communication middleware in distributed computing
3. Describe the synchronization and communication mechanisms in Distributed Computing.
4. Comprehend the distributed mutual exclusion algorithms and deadlocks in distributed systems
5. Understand the concept of cloud computing and virtualization.

**Course Contents:**

1. Introduction to Distributed Systems, Distributed Computing
2. Communication in Distributed Systems for Distributed Computing
3. Synchronization in Distributed systems
4. Distributed Mutual Exclusion and Deadlock Detection in Distributed Systems
5. Introduction to Cloud Computing

**Laboratory Exercises:**

1. Write a program to implement Echo server using socket programming
2. Design a distributed application using RMI.
3. Design a distributed application using RPC.
4. Write a C program to implement Lamport's Logical Clock Algorithm.
5. Write a C Program to Implement Bully Election Algorithm.
6. Implement a Network File System
7. Install VM-Ware Workstation on a windows or Ubuntu server platform as per requirement.
8. CASE STUDY: Applications of Cloud Computing
9. Create a resource for one of the cloud services like AWS Lambda or Azure Storage account, comprehend its usage and delete the same

**Assessment Scheme:**

**Class Continuous Assessment (CCA) :30 Marks**

**Laboratory Continuous Assessment (LCA): 30 Marks**

| Mid Term | Component 1 (Active Learning) | Component 2 |
|---|---|---|
| 15 Marks | 10 Marks | 5 Marks |
| **Practical Performance** | **Active learning / Mini Project/Additional implementation/ On paper design** | **End term practical /oral examination** |
| 10 Marks | 10 Marks | 10 Marks |

**Term End Examination : 40 Marks**

**Syllabus: Theory**

| Module No. | Contents | Workload in Hrs |
|---|---|---|
| | | Theory |
| 1 | **Introduction to Distributed Systems, Distributed Computing**<br>Definition, Characterization of distributed systems and Distributed Communication- Trends in distributed systems, Focus on resource sharing, Challenges.<br>Types of distributed systems.<br>System Models - Introduction, Physical models, Architectural models, Fundamental models. A Model of Distributed Computations. Distributed Shared Memory. | 08 |
| 2 | **Communication in Distributed Systems for Distributed Computing**<br>Primitives for Distributed Communication.<br>Introduction to middleware. Message Oriented Communication, TCP and UDP characteristics and differences, Multicast Communication.<br>Remote Methods Invocation[RMI] , Remote Procedure Call [RPC] | 08 |
| 3 | **Synchronization in Distributed Systems**<br>Clock synchronization -Physical Clocks, Logical Clocks.<br>Clock Synchronization Algorithms.Scalor time, Vector time.<br>Logical clocks - Lamport's Logical Clocks, Vector Clocks. Physical clock Synchronization- NTP. | 08 |

| | | |
|---|---|---|
| 4 | **Distributed Mutual Exclusion and Deadlock Detection in Distributed Systems.**<br>**Distributed Mutual Exclusion Algorithms**<br>A Centralized Algorithm, A Decentralized algorithm, Traditional Election Algorithms. Lamports Algorithm, Ricart-Agarwala Algorithm, Token Based Algorithms | 09 |
| | **Deadlocks in Distributed Systems**<br>System model, Models of Deadlocks, Chandy–Misra–Haas algorithm for the AND and OR model. Recovery mechanisms like Check pointing, Roll Back and Log based. Consensus and Agreement algorithms. | |
| 5 | **Introduction to Cloud Computing**<br>Introduction, Roots of Cloud Computing: From Mainframes to Cloud, Benefits of Cloud Computing SOA, Web services, Role of Networks in Cloud Computing: Cloud types and service models, Primary Cloud Service models, Cloud Services brokerage, Primary cloud deployment models, cloud computing reference model, The greenfield and brownfield deployment options, SLAs, Frequently used cloud services across industry, Basic serverless cloud architecture, Basic microservice based cloud architecture | 12 |

# UNIT I

## INTRODUCTION TO DISTRIBUTED SYSTEMS, DISTRIBUTED COMPUTING

Definition, Characterization of distributed systems and Distributed Communication- Trends in distributed systems, Focus on resource sharing, Challenges.
Types of distributed systems.
System Models - Introduction, Physical models, Architectural models, Fundamental models. A Model of Distributed Computations.
Distributed Shared Memory.

# 1. WHAT IS A DISTRIBUTED SYSTEM?

Definition: A *distributed system* is one in which **components** located at networked computers communicate and **coordinate** their actions only by passing **messages**.

**Distributed computing system :** A distributed computer system consists of multiple software components that are on multiple computers, but it runs as a single system. The computers that are in a distributed system can be physically close together and connected by a local network, or they can be geographically distant and connected by a wide area network.

Distributed Computing systems are many times known simply as Distributed Systems

**Distributed computing :** a method of computer processing in which different parts of a computer program are run on two or more computers that are communicating with each other over a network.

**Distributed Operating System :** Software for Distributed Systems

- Distributed computing refers to the use of distributed systems to complete computing tasks.

- It applies the principles of distributed systems to execute programs on different computer systems connected in a distributed system

# 1.1 CENTRALIZED SYSTEM CHARACTERISTICS

- One component with non-autonomous parts
- Component shared by users all the time
- All resources accessible
- Software runs in a single process
- Single point of control
- Single point of failure

# 1.2 DISTRIBUTED SYSTEM CHARACTERISTICS

- Multiple autonomous components
- Components are not shared by all users
- Resources may not be accessible
- Software runs in concurrent processes on different processors
- Multiple points of control
- Multiple points of failure

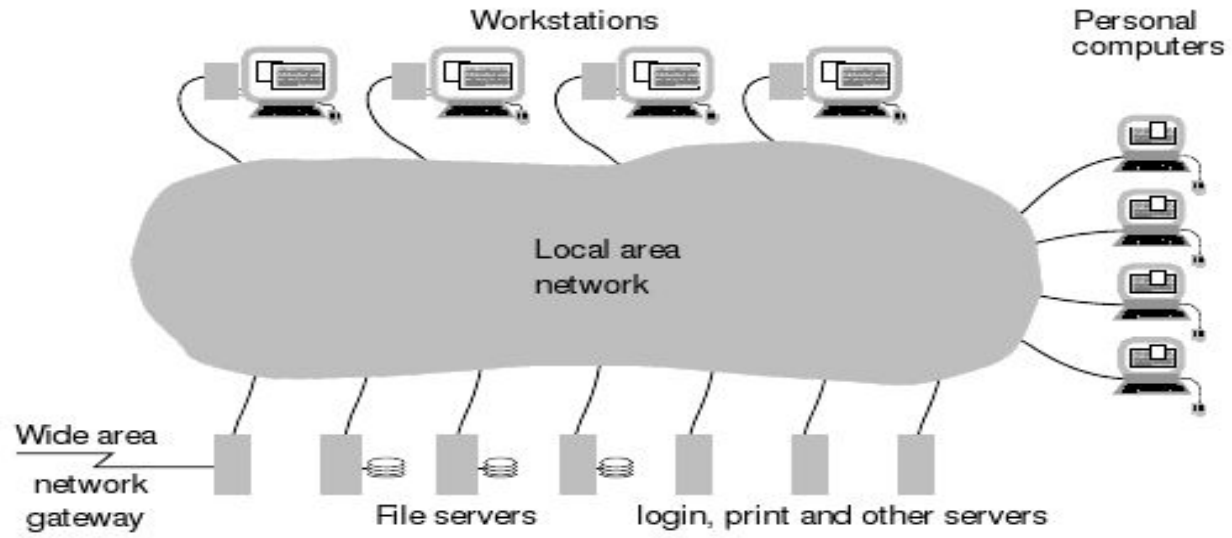
# 2. EXAMPLES OF DISTRIBUTED SYSTEMS

- Local Area Network and Intranet
- Database Management System
- Automatic Teller Machine Network
- Internet/World-Wide Web
- Mobile and Ubiquitous Computing

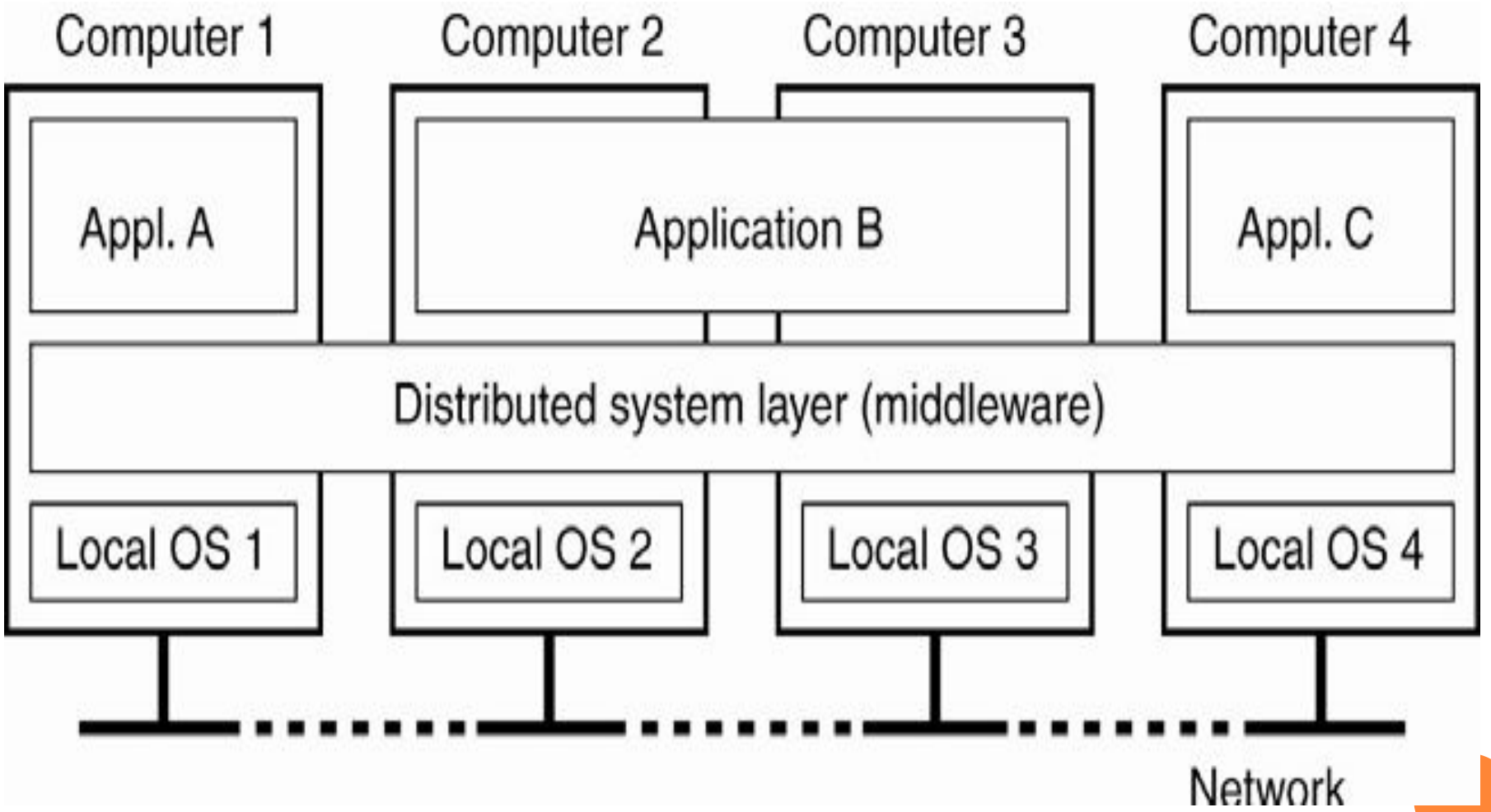
- A distributed system consists of a collection of autonomous computers linked by a computer network and equipped with a distributed software enables computers to coordinate their activities and to share resources of the systems.
- Distributed computing is a computing concept that, in its most general sense, refers to multiple computer systems working on a single problem.
- In distributed computing, a single problem is divided into many parts, and each part is solved by different computers.
- As long as the computers are networked, they can communicate with each other to solve the problem. If done properly, the computers perform like a single entity.

# A Distributed System

**Figure 1.1** A simple distributed system.



# A Distributed System



# 3. COMMON CHARACTERISTICS

- What are we trying to achieve when we construct a distributed system?
- Certain common characteristics can be used to assess distributed systems
  - Heterogeneity
  - Openness
  - Security
  - Scalability
  - Failure Handling
  - Concurrency
  - Transparency

# 3.1 HETEROGENEITY

- Variety and differences in
  - Networks
  - Computer hardware
  - Operating systems
  - Programming languages
  - Implementations by different developers
- *Middleware* as software layers to provide a programming abstraction as well as masking the heterogeneity of the underlying networks, hardware, OS, and programming languages (e.g., CORBA).
- *Mobile Code* to refer to code that can be sent from one computer to another and run at the destination (e.g., Java applets and Java *virtual machine*).

# 3.2 OPENNESS

- Openness is concerned with extensions and improvements of distributed systems.
- Detailed interfaces of components need to be published.
- New components have to be integrated with existing components.
- Differences in data representation of interface types on different processors (of different vendors) have to be resolved.

# 3.3 SECURITY

- In a distributed system, clients send requests to access data managed by servers, resources in the networks:
  - Doctors requesting records from hospitals
  - Users purchase products through electronic commerce
- Security is required for:
  - Concealing the contents of messages: security and privacy
  - Identifying a remote user or other agent correctly (authentication)
- New challenges:
  - Denial of service attack
  - Security of mobile code

# 3.4 SCALABILITY

- Adaptation of distributed systems to
  - accommodate more users
  - respond faster (this is the hard one)
- Usually done by adding more and/or faster processors.
- Components should not need to be changed when scale of a system increases.
- Design components to be scalable!

# 3.5 FAILURE HANDLING (FAULT TOLERANCE)

- Hardware, software and networks fail!
- Distributed systems must maintain *availability* even at low levels of hardware/software/network *reliability*.
- Fault tolerance is achieved by
  - recovery
  - redundancy

# 3.6 CONCURRENCY

- Components in distributed systems are executed in concurrent processes.
- Components access and update shared resources (e.g. variables, databases, device drivers).
- Integrity of the system may be violated if concurrent updates are not coordinated.
  - Lost updates
  - Inconsistent analysis

# 3.7 TRANSPARENCY

- Distributed systems should be perceived by users and application programmers as a whole rather than as a collection of cooperating components.
- Transparency has different aspects.
- These represent various properties that distributed systems should have.

# 3.7.1 ACCESS TRANSPARENCY

- Enables local and remote information objects to be accessed using identical operations.
- Example: File system operations in NFS.
- Example: Navigation in the Web.
- Example: SQL Queries

# 3.7.2 LOCATION TRANSPARENCY

- Enables information objects to be accessed without knowledge of their location.
- Example: File system operations in NFS
- Example: Pages in the Web
- Example: Tables in distributed databases

# 3.7.3 CONCURRENCY TRANSPARENCY

- Enables several processes to operate concurrently using shared information objects without interference between them.
- Example: NFS
- Example: Automatic teller machine network
- Example: Database management system

# 3.7.4 REPLICATION TRANSPARENCY

- Enables multiple instances of information objects to be used to increase reliability and performance without knowledge of the replicas by users or application programs
- Example: Distributed DBMS
- Example: Mirroring Web Pages.

# 3.7.5 FAILURE TRANSPARENCY

- Enables the concealment of faults
- Allows users and applications to complete their tasks despite the failure of other components.
- Example: Database Management System

# 3.7.6 MOBILITY TRANSPARENCY

- Allows the movement of information objects within a system without affecting the operations of users or application programs

- Example: NFS

- Example: Web Pages

# 3.7.7 PERFORMANCE TRANSPARENCY

- Allows the system to be reconfigured to improve performance as loads vary.

- Example: Distributed make.

# 3.7.8 SCALING TRANSPARENCY

- Allows the system and applications to expand in scale without change to the system structure or the application algorithms.
- Example: World-Wide-Web
- Example: Distributed Database

# 4. BASIC DESIGN ISSUES

- General software engineering principles include rigor and formality, separation of concerns, modularity, abstraction, anticipation of change, …
- Specific issues for distributed systems:
  - Naming
  - Communication
  - Software structure
  - System architecture
  - Workload allocation
  - Consistency maintenance

# Distributed Systems: Challenges/Failures

- There are also multiple challenges of distributed systems that determine the performance of the overall system. **Heterogeneity**
- Heterogeneity is one of the challenges of a distributed system that refers to differences in hardware, software, or network configurations among nodes. This can present challenges for communication and coordination.

Techniques for managing heterogeneity include middleware, virtualization, standardization, and service-oriented architecture. These approaches can help build robust and scalable systems that accommodate diverse configurations.

## Scalability

- Scalability is one of the challenges in distributed systems. As distributed systems grow in size and complexity, it becomes increasingly difficult to maintain their performance and availability. The major challenges are security, maintaining consistency of data in every system, network latency between systems, resource allocation, or proper node balancing across multiple nodes.

## Openness

- Openness in distributed systems refers to achieving a standard between different systems that use different standards, protocols, and data formats. It is crucial to ensure that different systems can communicate and exchange data seamlessly without the need for extensive manual intervention. It is also important to maintain the correct amount of transparency and security in such systems.

## Transparency

- Transparency refers to the level of abstraction present in the system to hide complex information from the user. It is essential to ensure that failures are transparent to users and do not affect the overall system's performance. Systems with different hardware and software configurations provide to be a challenge for Transparency. Security is also a concern to maintain transparency in distributed systems.

## Concurrency

- Concurrency is the ability to process data parallelly on different nodes of the system. One of the primary challenges of concurrency in distributed systems is the issue of race conditions. Problems like communication and synchronization between nodes also pose a challenge. When a node fails, the fault tolerance mechanism must ensure synchronization.

## Security

- The distributed and heterogeneous nature of the distributed system makes security a major challenge for data processing systems. The system must ensure confidentiality from unauthorized access as data is transmitted across multiple nodes. Various methods like Digital signatures, Checksums, and Hash functions should be used to verify the integrity of data as data is being modified by multiple systems. Authentication mechanisms are also challenging as users and processes may be located on different nodes.

- **Failure Handling**
- One of the primary challenges of failure handling in distributed systems is identifying and diagnosing failures as failure can occur at any node. Logging mechanisms should be implemented to identify the failed nodes. Techniques like redundancy, replication, and checkpoints should be used to ensure the continuous working of the system in case of a node failure. Data recovery should be implemented with techniques like Rollback to recover data in the event of a failure.

# Importance of Resource Sharing in Distributed Systems

- Resource sharing in distributed systems is of paramount importance for several reasons:
- **Efficiency and Cost Savings**: By sharing resources like storage, computing power, and data, distributed systems maximize utilization and minimize waste, leading to significant cost reductions.
- **Scalability**: Distributed systems can easily scale by adding more nodes, which share the workload and resources, ensuring the system can handle increased demand without a loss in performance.

- **Reliability and Redundancy**: Resource sharing enhances system reliability and fault tolerance. If one node fails, other nodes can take over, ensuring continuous operation.
- **Collaboration and Innovation**: Resource sharing facilitates collaboration among geographically dispersed teams, fostering innovation by providing access to shared tools, data, and computational resources.
- **Load Balancing**: Efficient distribution of workloads across multiple nodes prevents any single node from becoming a bottleneck, ensuring balanced performance and preventing overloads.

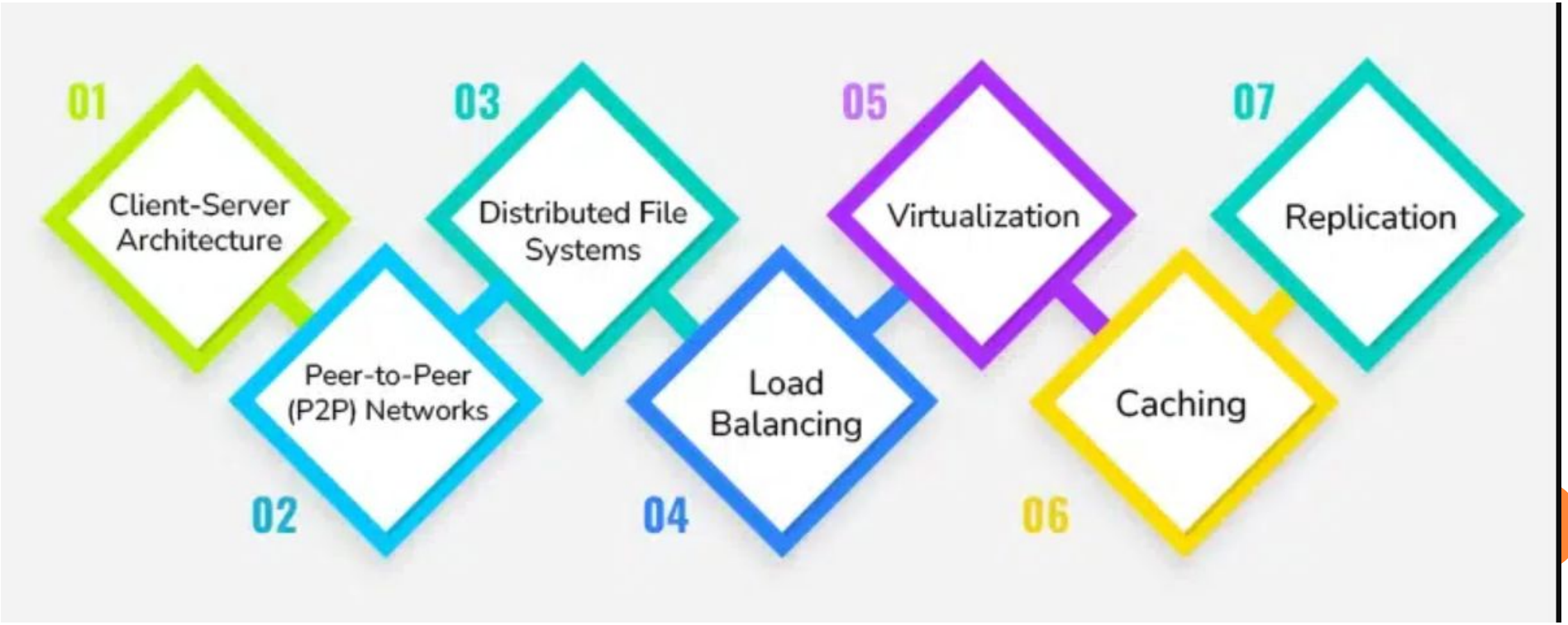
# Types of Resources in Distributed Systems

- **Computational Resources**: These include CPU cycles and processing power, which are shared among multiple users and applications to perform various computations and processing tasks.
- **Storage Resources**: Distributed storage systems allow data to be stored across multiple nodes, ensuring data availability, redundancy, and efficient access.
- **Memory Resources**: Memory can be distributed and shared across nodes, allowing applications to utilize a larger pool of memory than what is available on a single machine.
- **Network Resources**: These include bandwidth and network interfaces, which facilitate communication and data transfer between nodes in a distributed system.

- **Data Resources**: Shared databases, files, and data streams that are accessible by multiple users and applications for reading and writing operations.
- **Peripheral Devices**: Devices such as printers, scanners, and specialized hardware that can be accessed remotely within the distributed network.

# Resource Sharing Mechanisms

- Resource sharing in distributed systems is facilitated through various mechanisms designed to optimize utilization, enhance collaboration, and ensure efficiency. Some common mechanisms include:



1. **Client-Server Architecture**: A classic model where clients request services or resources from centralized servers. This architecture centralizes resources and services, providing efficient access but potentially leading to scalability and reliability challenges.
2. **Peer-to-Peer (P2P) Networks**: Distributed networks where each node can act as both a client and a server. P2P networks facilitate direct resource sharing between nodes without reliance on centralized servers, promoting decentralized and scalable resource access.
3. **Distributed File Systems**: Storage systems that distribute files across multiple nodes, ensuring redundancy and fault tolerance while allowing efficient access to shared data.

4. **Load Balancing**: Mechanisms that distribute workload across multiple nodes to optimize resource usage and prevent overload on individual nodes, thereby improving performance and scalability.
5. **Virtualization**: Techniques such as virtual machines (VMs) and containers that abstract physical resources, enabling efficient resource allocation and utilization across distributed environments.
6. **Caching**: Storing frequently accessed data closer to users or applications to reduce latency and improve responsiveness, enhancing overall system performance.
7. **Replication**: Creating copies of data or resources across multiple nodes to ensure data availability, fault tolerance, and improved access speed.

# Best Architectures for Resource Sharing in Distributed System

- The best architectures for resource sharing in distributed systems depend on the specific requirements and characteristics of the system. Here are some commonly adopted architectures that facilitate efficient resource sharing:

1. **Client-Server Architecture**:
    1. **Advantages**: Centralized management simplifies resource allocation and access control. It is suitable for applications where clients primarily consume services or resources from centralized servers.
    2. **Use Cases**: Web applications, databases, and enterprise systems where centralized control and management are critical.

2. **Peer-to-Peer (P2P) Architecture**:
    2. **Advantages**: Decentralized nature facilitates direct resource sharing between peers without dependency on centralized servers, enhancing scalability and fault tolerance.
    3. **Use Cases**: File sharing, content distribution networks (CDNs), and collaborative computing environments.
3. **Service-Oriented Architecture (SOA)**:
    2. **Advantages**: Organizes services as reusable components that can be accessed and shared across distributed systems, promoting interoperability and flexibility.
    3. **Use Cases**: Enterprise applications, where modular services such as authentication, messaging, and data access are shared across different departments or systems.

4. **Microservices Architecture**:
    4. **Advantages**: Decomposes applications into small, independent services that can be developed, deployed, and scaled independently. Each microservice can share resources selectively, optimizing resource usage.
    5. **Use Cases**: Cloud-native applications, where scalability, agility, and resilience are paramount.
5. **Distributed File System Architecture**:
    4. **Advantages**: Distributes file storage across multiple nodes, providing redundancy, fault tolerance, and efficient access to shared data.
    5. **Use Cases**: Large-scale data storage and retrieval systems, such as Hadoop Distributed File System (HDFS) for big data processing.

# Resource Allocation Strategies in Distributed System

- Here are some common strategies:
- **1. Static Allocation:**
- **Description**: Resources are allocated based on fixed, predetermined criteria without considering dynamic workload changes.
- **Advantages**: Simple to implement and manage, suitable for predictable workloads.
- **Challenges**: Inefficient when workload varies or when resources are underutilized during low-demand periods.

- **2. Dynamic Allocation:**
- **Description**: Resources are allocated based on real-time demand and workload conditions.
- **Advantages**: Maximizes resource utilization by adjusting allocations dynamically, responding to varying workload patterns.
- **Challenges**: Requires sophisticated monitoring and management mechanisms to handle dynamic changes effectively.
- **3. Load Balancing:**
- **Description**: Distributes workload evenly across multiple nodes or resources to optimize performance and prevent overload.
- **Strategies**: Round-robin scheduling, least connection method, and weighted distribution based on resource capacities.
- **Advantages**: Improves system responsiveness and scalability by preventing bottlenecks.
- **Challenges**: Overhead of monitoring and adjusting workload distribution.

- **4. Reservation-Based Allocation:**
- **Description**: Resources are reserved in advance based on anticipated future demand or specific application requirements.
- **Advantages**: Guarantees resource availability when needed, ensuring predictable performance.
- **Challenges**: Potential resource underutilization if reservations are not fully utilized.
- **5. Priority-Based Allocation:**
- **Description**: Assigns priorities to different users or applications, allowing higher-priority tasks to access resources before lower-priority tasks.
- **Advantages**: Ensures critical tasks are completed promptly, maintaining service-level agreements (SLAs).
- **Challenges**: Requires fair prioritization policies to avoid starvation of lower-priority tasks.

## Challenges in Resource Sharing in Distributed System

- **Consistency and Coherency**: Ensuring that shared resources such as data or files remain consistent across distributed nodes despite concurrent accesses and updates.
- **Concurrency Control**: Managing simultaneous access and updates to shared resources to prevent conflicts and maintain data integrity.
- **Fault Tolerance**: Ensuring resource availability and continuity of service in the event of node failures or network partitions.
- **Scalability**: Efficiently managing and scaling resources to accommodate increasing demands without compromising performance.

- **Load Balancing**: Distributing workload and resource usage evenly across distributed nodes to prevent bottlenecks and optimize resource utilization.
- **Security and Privacy**: Safeguarding shared resources against unauthorized access, data breaches, and ensuring privacy compliance.
- **Communication Overhead**: Minimizing overhead and latency associated with communication between distributed nodes accessing shared resources.
- **Synchronization**: Coordinating activities and maintaining synchronization between distributed nodes to ensure consistent and coherent resource access.

# SYSTEMS MODELS

*

**Architectural Models** (describing components ,and their relationship)

- Client server models
- Peer process models

**Fundamental Models**

- Interaction Model
- Failure Model
- Security Model

(formal description of properties common in all architectural models)



# ARCHITECTURAL ELEMENTS

- What are the entities
- How do they communicate, or, more specifically, what *communication paradigm* is used?
- What (potentially changing) roles and responsibilities do they have in the overall architecture?
- How are they mapped on to the physical distributed infrastructure (what is their *placement)?*



# COMMUNICATING ENTITIES

- From a system perspective, entities are typically *processes*.
  - In some primitive environments, such as sensor networks, the entities are *nodes*.
  - In most distributed system environments, processes are supplemented **by *threads,*** that are the endpoints of communication



- *Objects:* Objects have been introduced to enable and encourage the use of object oriented approaches in distributed systems

- *Components:*

- *Web services:*

- **Communication paradigms**
  - Inter-process communication - low-level support for communication between processes in distributed systems by message passing.
  - Remote invocation - the most common communication paradigm in distributed systems.

- *Request-reply protocols* - *M*essages from client to server and then from server back to client, containing encoding operation and holding associated arguments.

- *Remote procedure calls* - The underlying RPC system then hides important aspects of distribution, including the encoding and decoding of parameters

- *Remote method invocation*



# Architectural models

*

- Simplification by distinction between client, server and peer processes
  - Assessment of workload (responsibilities)
  - Failure impact analysis

- More dynamic systems can be built as variations on the client-server model
  - Moving code (from one process to another process)
    - Ex:: Client can download code from server & run it locally.
    - Code & objects are moved to reduce access delays & minimize comm. Traffic.

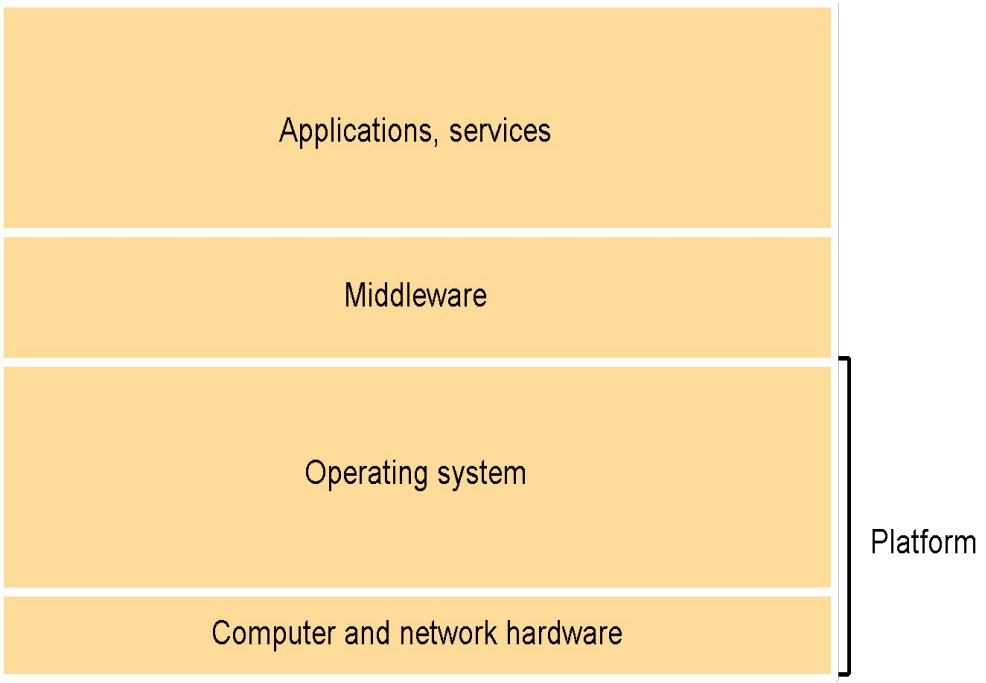
  - Adding/removing nodes or components
    - Discovery and advertisement of services

    - (Some DS enable comp & other mobile devices to be added or removed seamlessly, & allowed them to discover the available services & to offer their services to others.)

# Service Layers

- Software architecture
  - Structuring software as layers or modules
  - Services offered and requested by processes on same or different computers (More recently)
- **Service layers::**

  A distributed service can be provided by one or more interacting server processes
  - Ex:: Network Time Protocol




Software and Hardware service layers in DS

# Service Layers

*

- Middleware::

  - Masks heterogeneity & provide convenient prog. Model to application programmers.
  - Represented by processes or objects in a set of computers that interact with each other to implement communication & resource sharing,
  - To Implement communication and resource sharing
    - RMI
    - Group communication
  - Supplies application programming interface
  - **Provides**
    - Building blocks for software components



# Service Layers: Middleware

*

- **Object oriented Middleware products & standards are::**
  - Examples:
    - Sun RPC
      - Simple, remote procedure calling package
    - CORBA(Common Object Request Broker Architecture)
      - CORBA is useful because it enables separate pieces of software written in different languages and running on different computers to work with each other like a single application or set of services.
      - SERVICES:--Namming,Security,event notification,persistant storage, transactions.
    - Microsoft DCOM(Distributed Component Object Model)
      - is a proprietaryis a proprietary Microsoftis a proprietary Microsoft technology for communication among software componentsis a proprietary Microsoft technology for communication among software components distributed across networked computers.
    - Java RMI

# Service Layers: Middleware

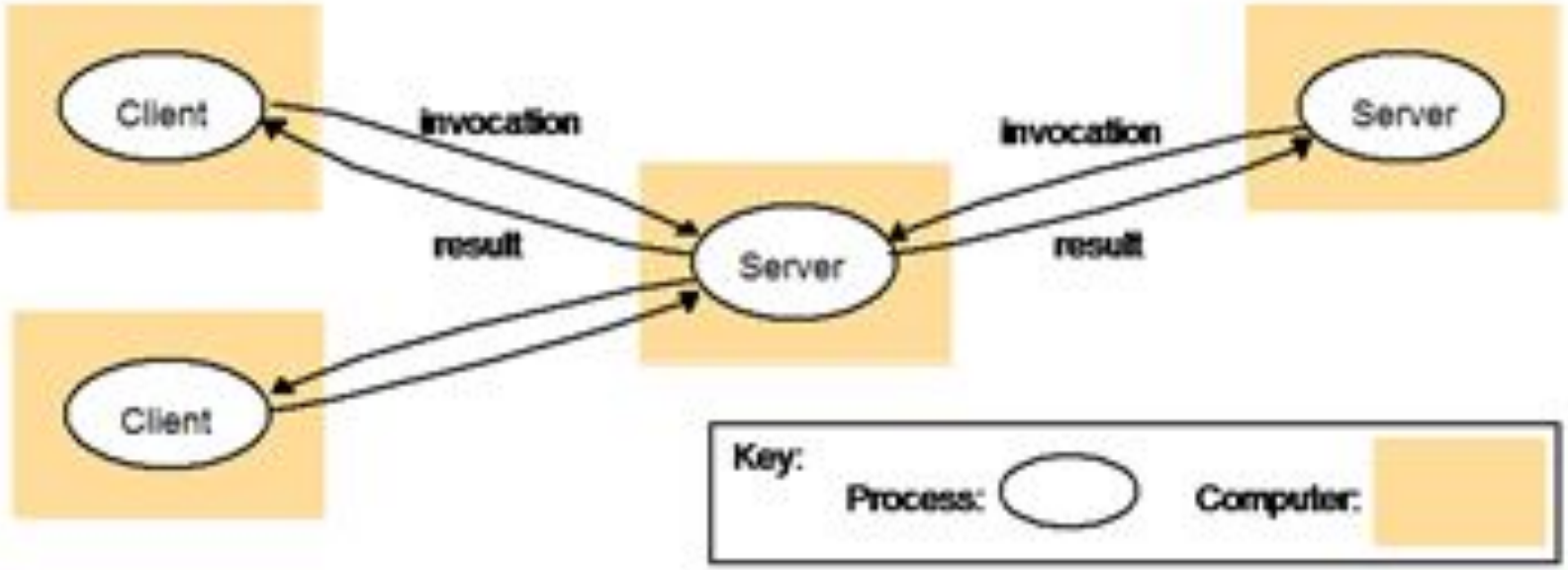
*

- Middleware services
  - Naming
  - Security
  - Transactions
  - Persistent storage
  - Event notification
- Limitations
  - Many distributed applications rely entirely on the services provided by the available middleware to support their needs for communication & data sharing
    - TCP reliable for packets (basic error correction and detection)
    - Mail transfer service increase fault tolerance (maintain record of progress)
    - Some functions can only be correctly implemented with the help of application level information
      - **Correct behaviour in distributed programs depends upon checks, error-correction mechanisms & security mechanism at various levels**

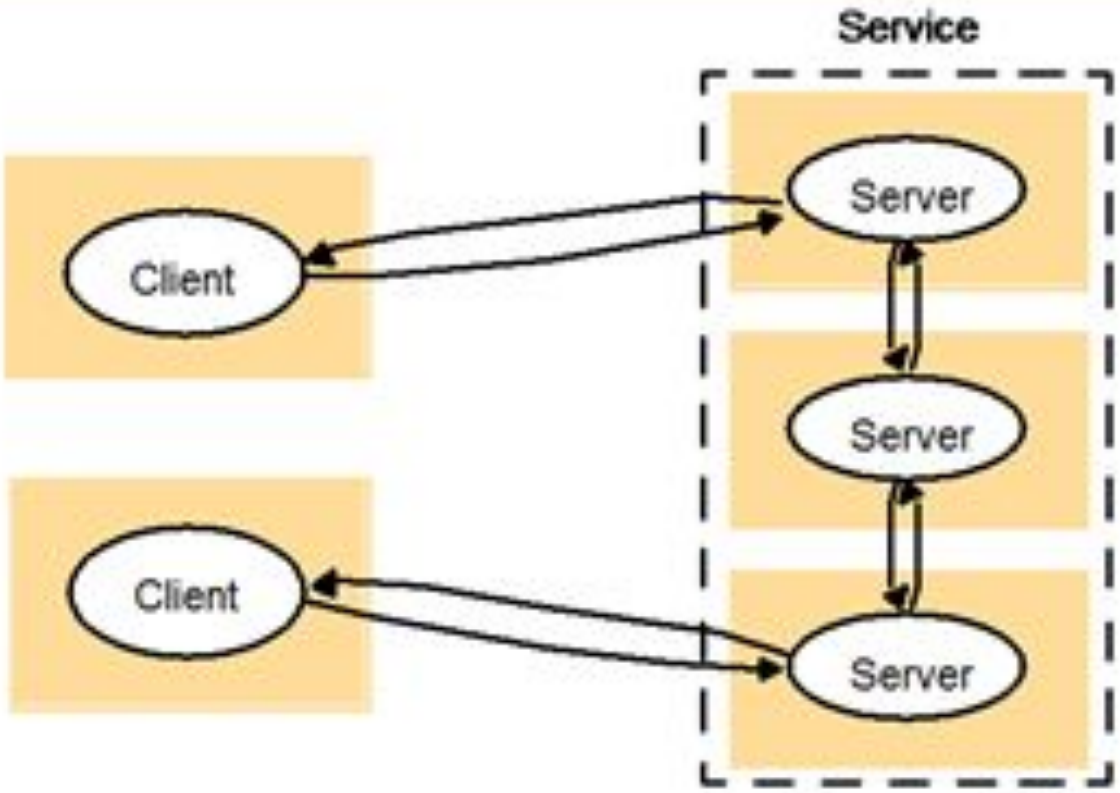


*

## Basic architectural models – client-server

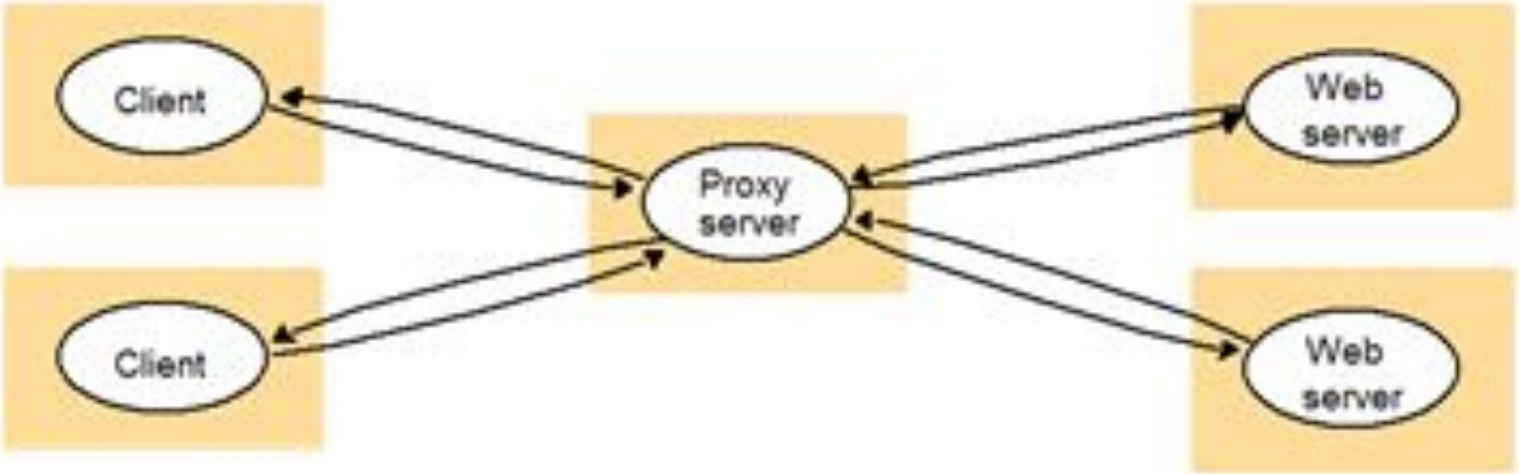


## Basic architectural models – multiple servers




Partition set of objects or
replicate set of objects

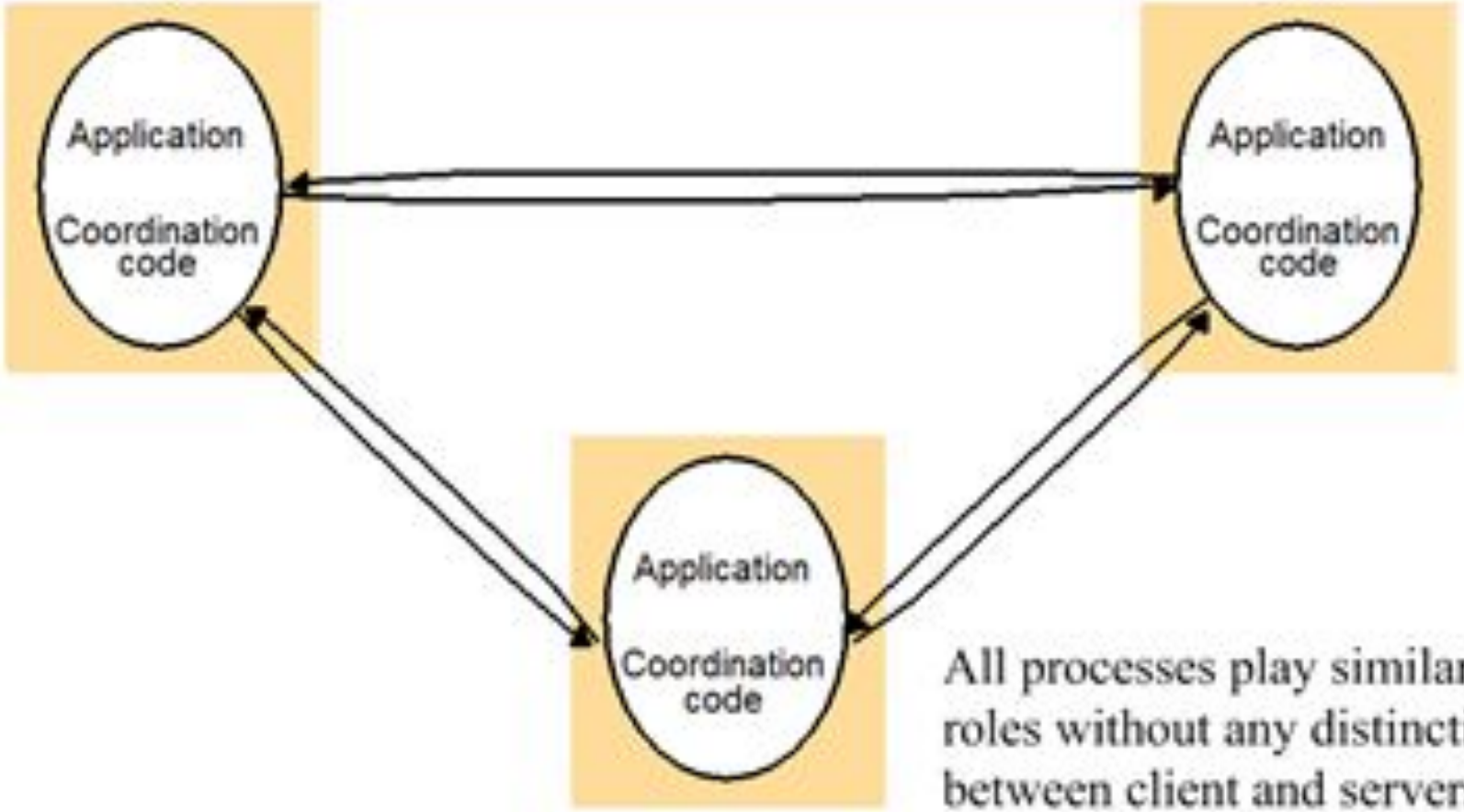
## Basic architectural models – proxy server and caching




Cache: Store of recently used data objects closer than the objects themselves

Proxy servers increase performance and availability

## Basic architectural models – peer processes




All processes play similar roles without any distinctio between client and servers.

# Variations on client server model

- Use of mobile code and mobile agents

- To be able to add and remove mobile devices in a convenient manner.



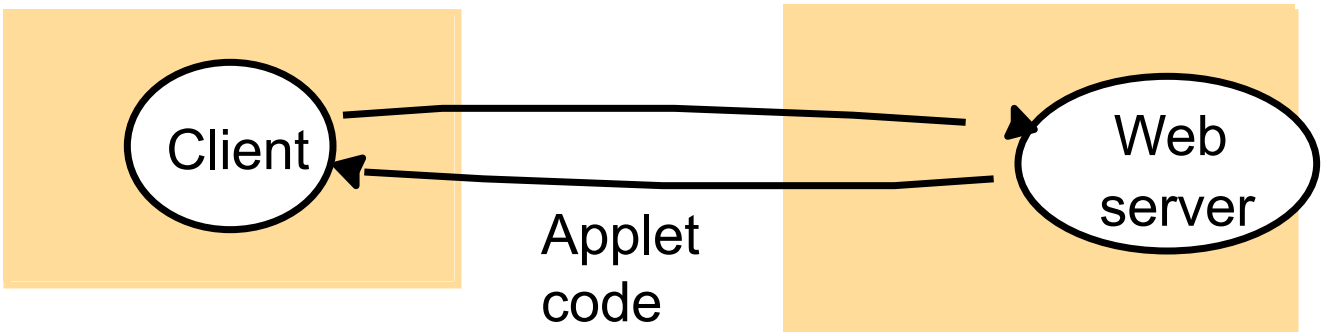
# Mobile code & Mobile agents

- **Mobile code**
- eg applets
- Only the code can be accessed
- **Mobile agents**
- It's a running program that includes

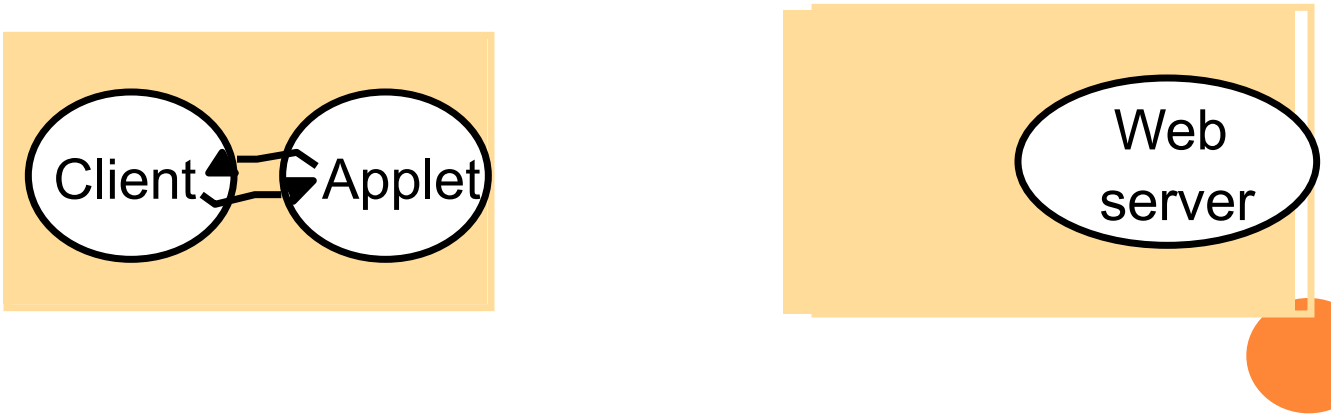
  Both code + data



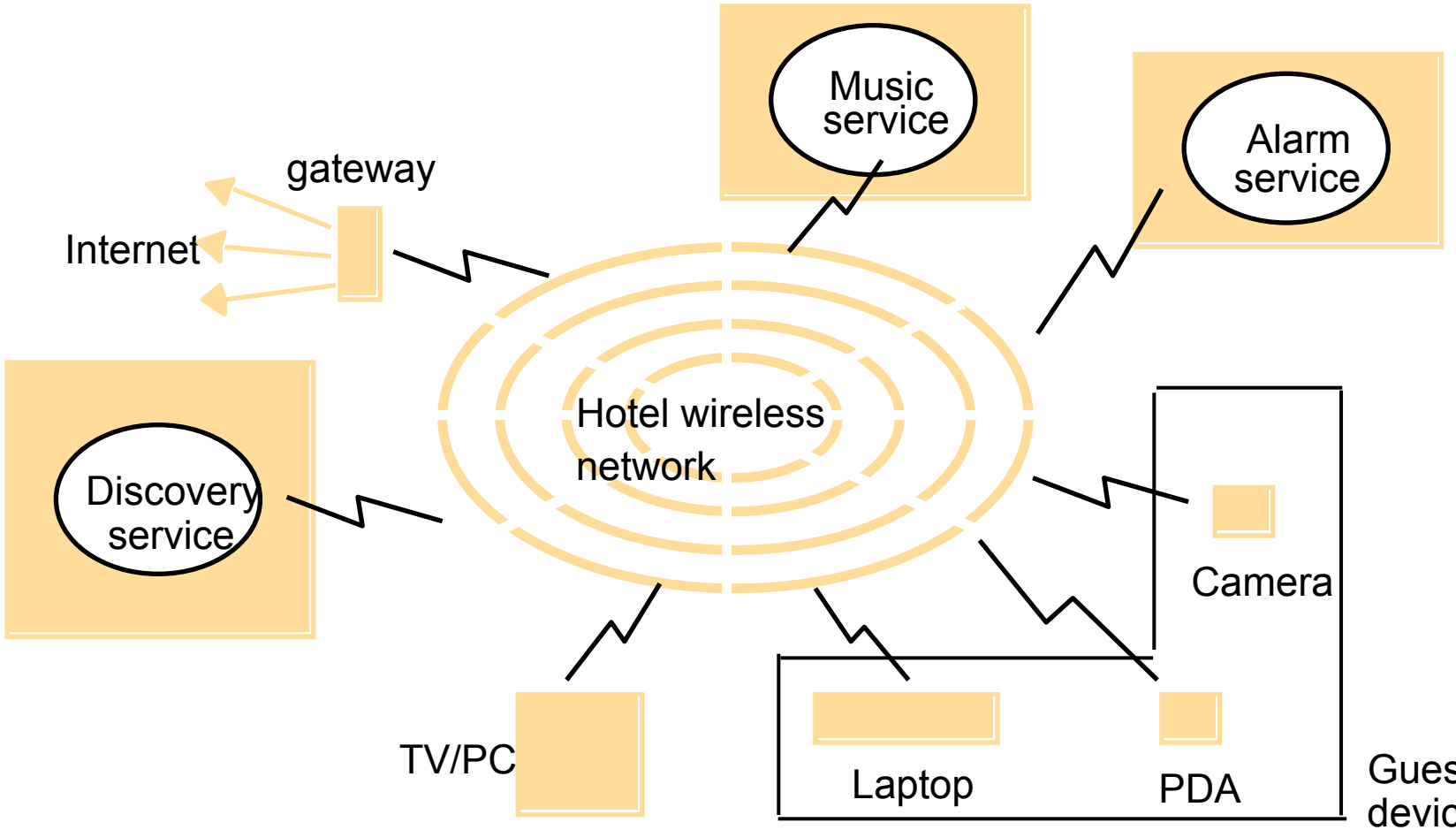
# Web applets




b) client interacts with the applet



# Mobile Devices and Spontaneous networking



# Mobile Devices and Spontaneous networking



- Key features
  - Easy connection to local network
    - Transparently reconfigured devices
  - Easy integration with local services
    - Autonomous service discoveries
- Issues
  - Internet addressing, and routing assumption
    - computers are at fixed locations
  - Limited connectivity
  - Security and privacy
- Discovery services
  - Registration service
  - Lookup service



# FUNDAMENTAL MODELS

- Interaction model
- Failure model
- Security model



# 4.1 NAMING

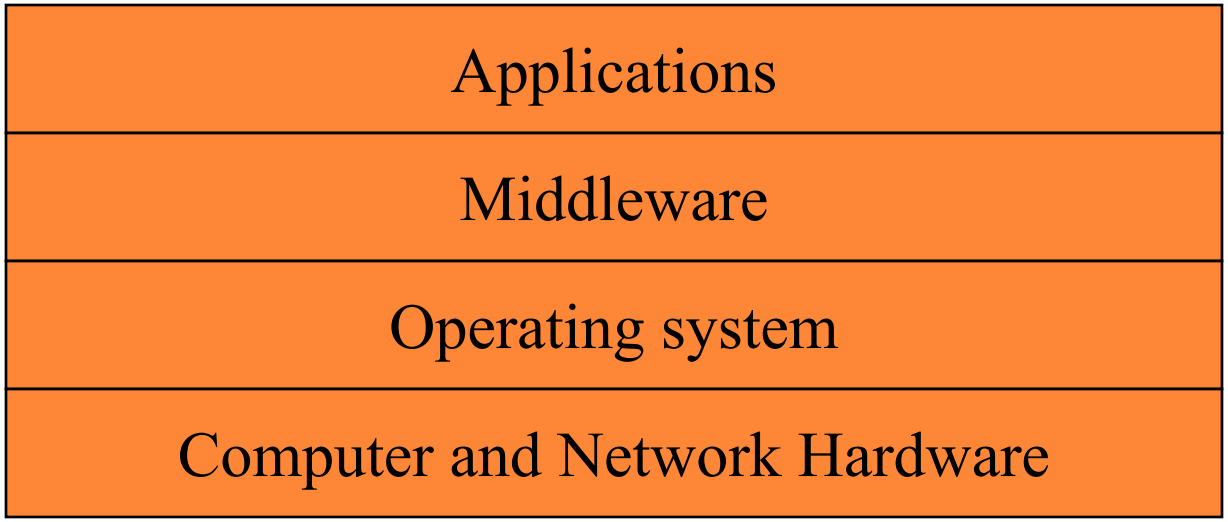
- A name is resolved when translated into an interpretable form for resource/object reference.
  - Communication identifier (IP address + port number)
  - Name resolution involves several translation steps
- Design considerations
  - Choice of name space for each resource type
  - Name service to resolve resource names to comm. id.
- Name services include naming context resolution, hierarchical structure, resource protection

# 4.2 COMMUNICATION

- Separated components communicate with sending processes and receiving processes for *data transfer* and *synchronization*.
- Message passing: *send* and *receive* primitives
  - synchronous or blocking
  - asynchronous or non-blocking
  - Abstractions defined: channels, sockets, ports.
- Communication patterns: client-server communication (e.g., RPC, function shipping) and group multicast

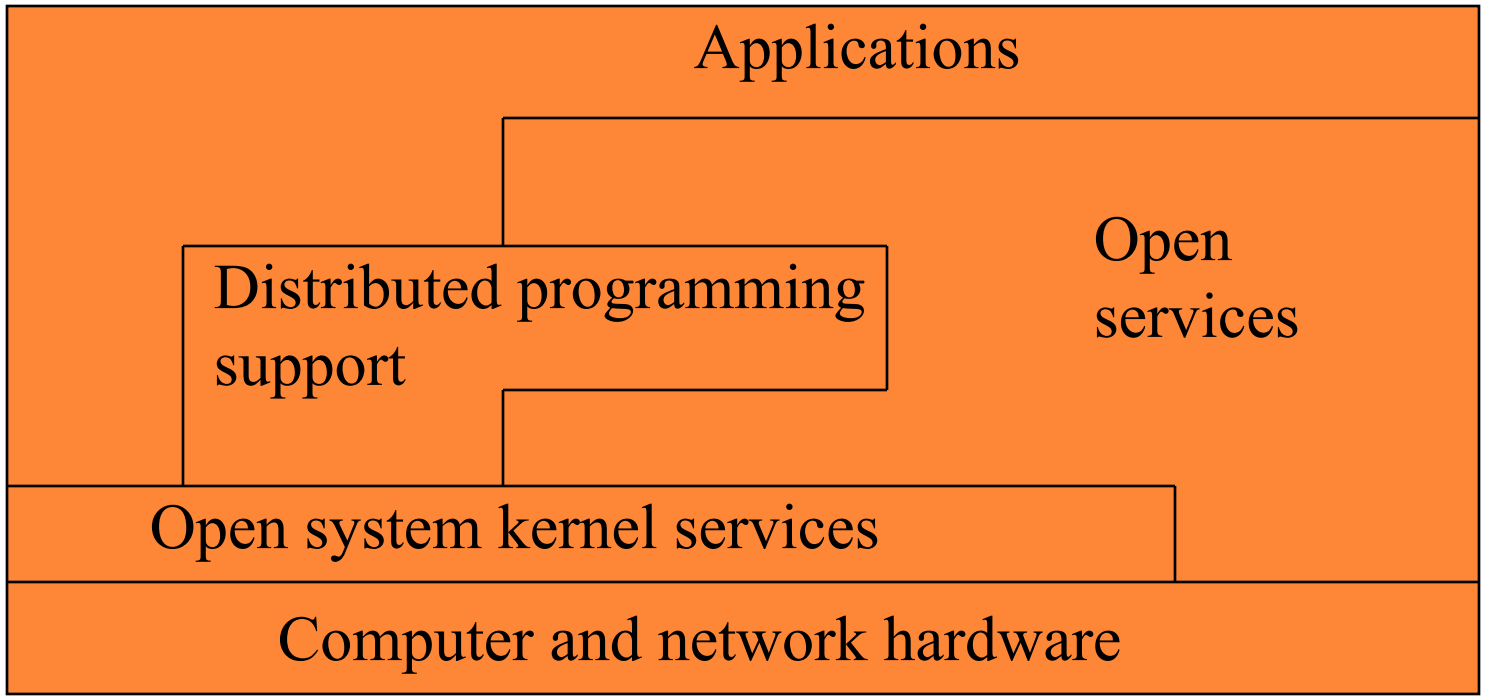
# 4.3 SOFTWARE STRUCTURE

- Layers in centralized computer systems:



# 4.3 SOFTWARE STRUCTURE

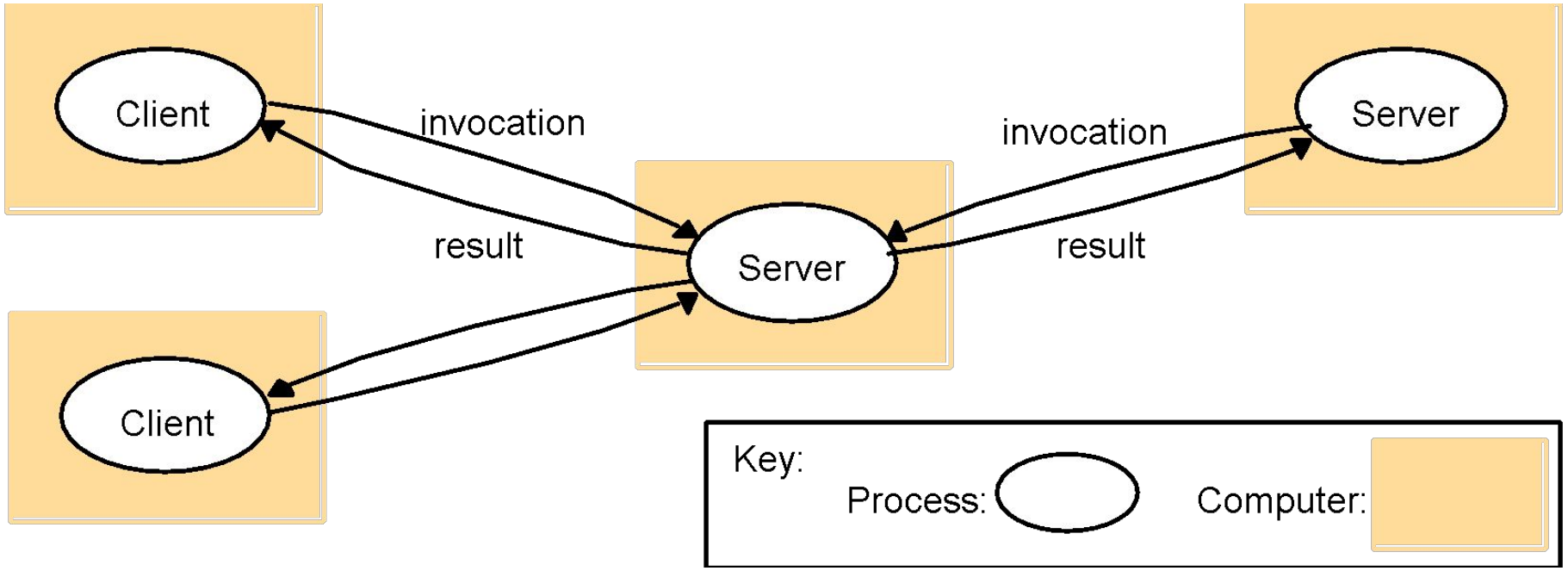
- Layers and dependencies in distributed systems:



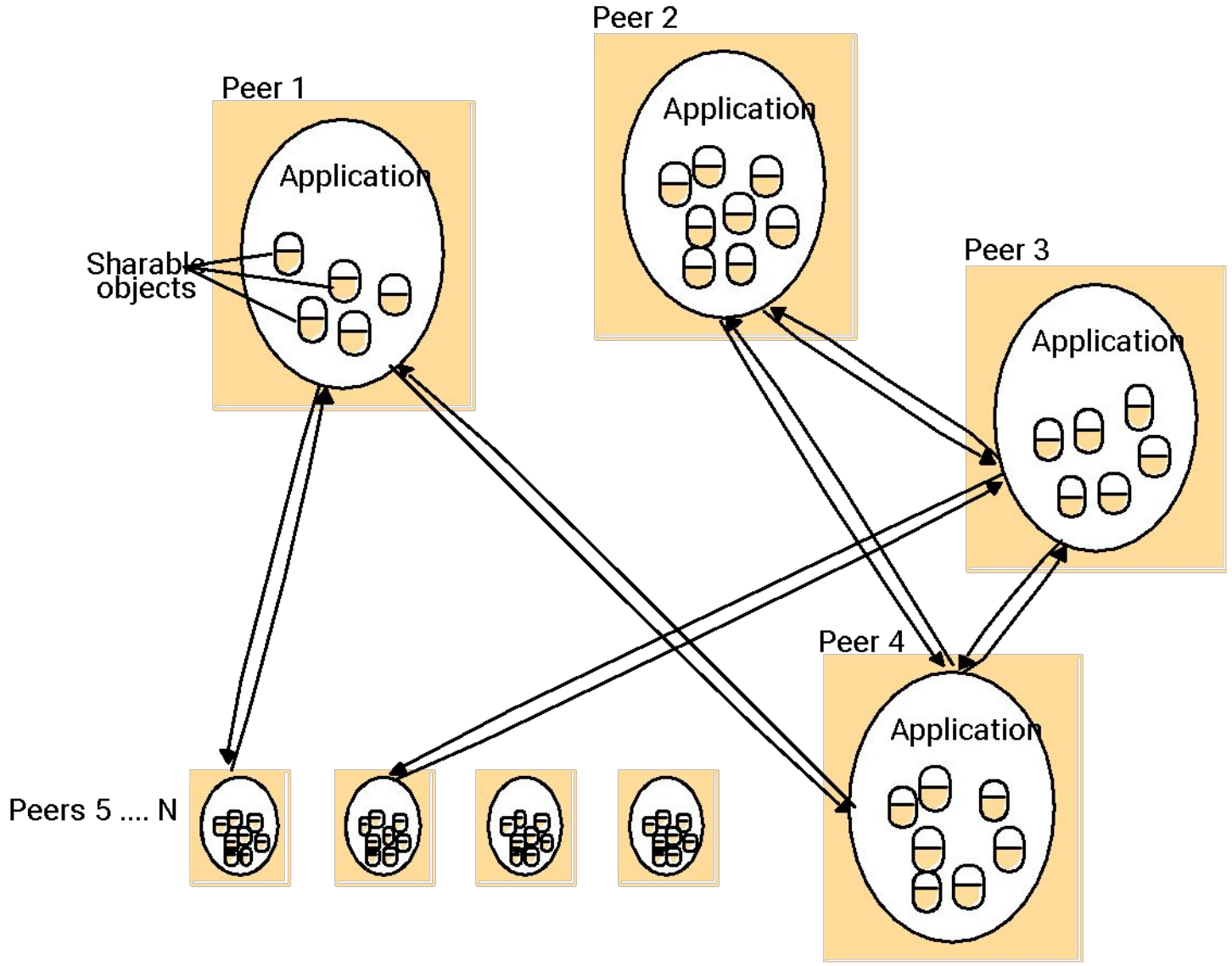
# 4.4 SYSTEM ARCHITECTURES

- Client-Server
- Peer-to-Peer
- Services provided by multiple servers
- Proxy servers and caches
- Mobile code and mobile agents
- Network computers
- Thin clients and mobile devices

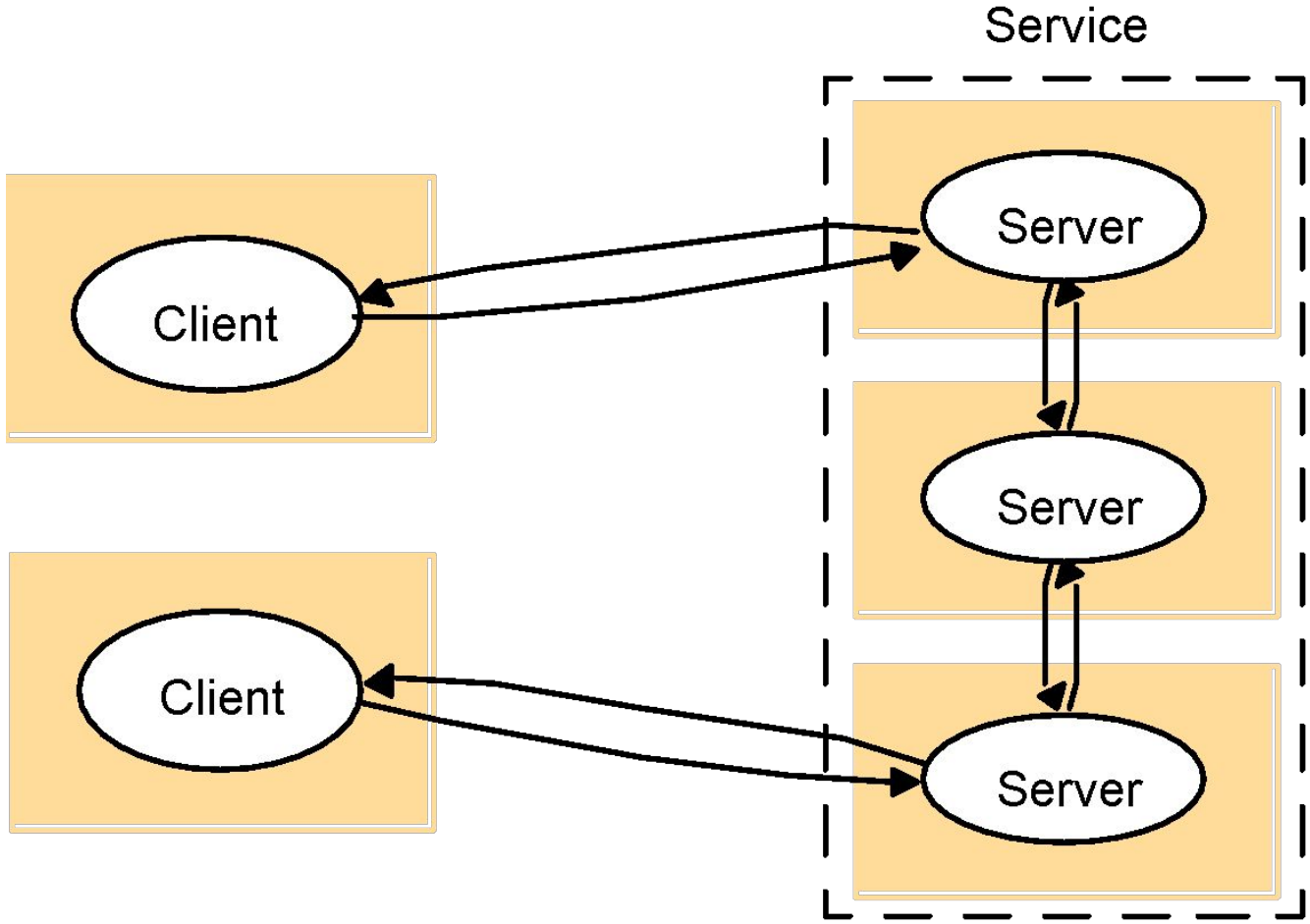
# 4.4.1 CLIENTS INVOKE INDIVIDUAL SERVERS



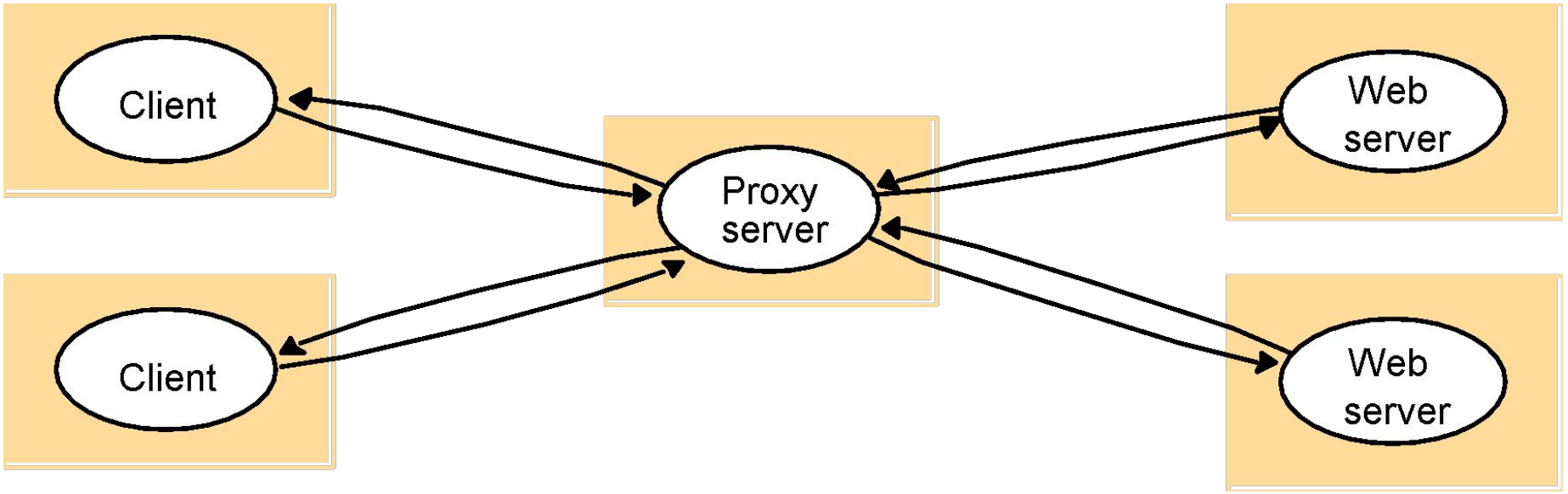
# 4.4.2 PEER-TO-PEER SYSTEMS



# 4.4.3 A SERVICE BY MULTIPLE SERVERS

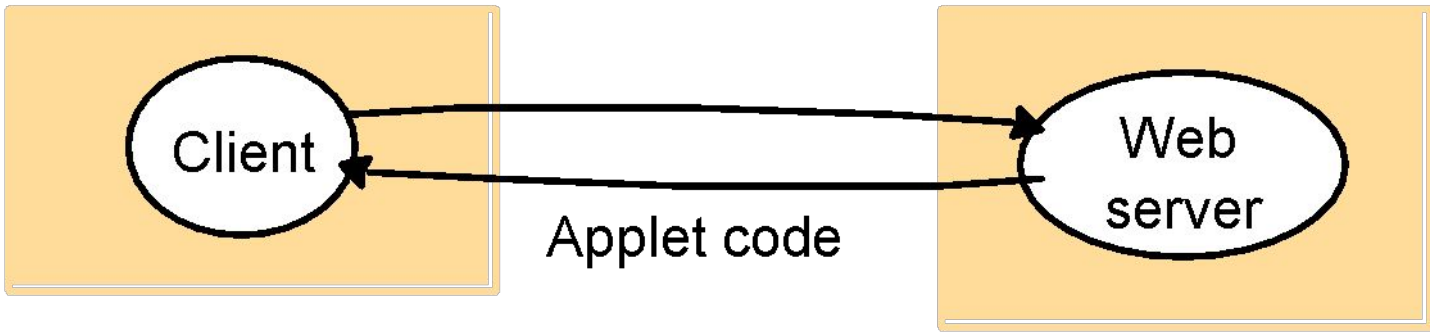


# 4.4.4 WEB PROXY SERVER



# 4.4.5 WEB APPLETS

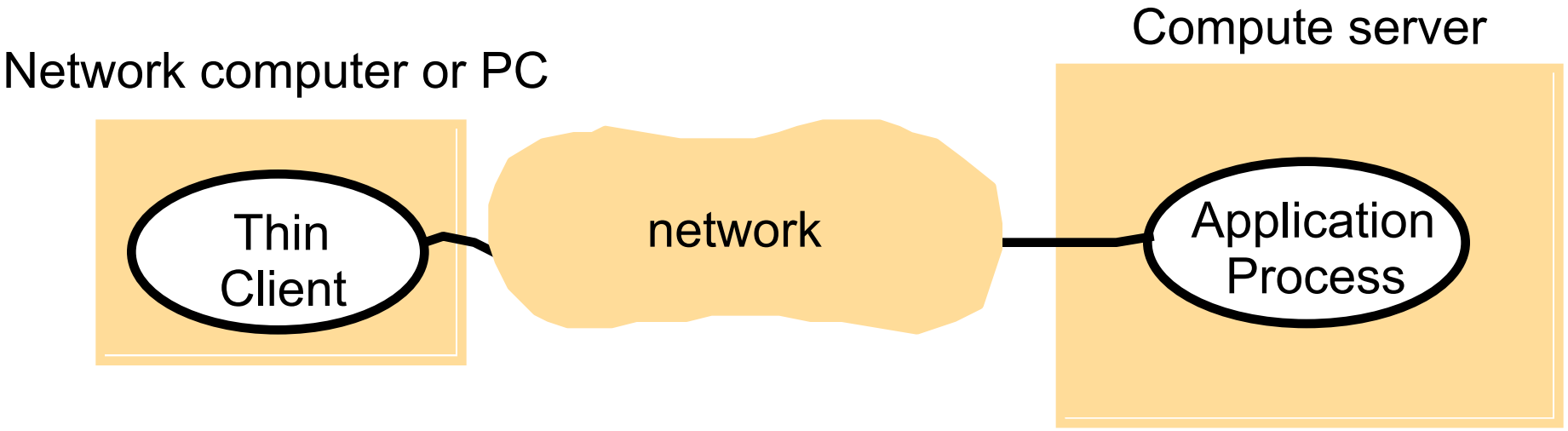
a) client request results in the downloading of applet code




b) client interacts with the applet



# 4.4.6 THIN CLIENTS AND COMPUTE SERVERS



# FUNDAMENTAL MODELS

- Interaction model
- Computation occurs within processes; the processes interact by passing messages, resulting in communication (information flow) and coordination (synchronization and ordering of activities) between processes. In the analysis and design of distributed systems we are concerned especially with these interactions.
- The interaction model must reflect the facts that communication takes place with delays that are often of considerable duration, and that the accuracy with which independent processes can be coordinated is limited by these delays and by the difficulty of maintaining the same notion of time across all the computers in a distributed system
- Failure model
- Our model defines and classifies the faults. This provides a basis for the analysis of their potential effects and for the design of systems that are able to tolerate faults of each type while continuing to run correctly.

# Security model

- The modular nature of distributed systems and their openness exposes them to attack by both external and internal agents.
- Our security model defines and classifies the forms that such attacks may take, providing a basis for the analysis of threats to a system and for the design of systems that are able to resist them.

# INTERACTION MODEL

- Performance of communication channels
- Computer clocks and timing events
- Two variants of the interaction model
- Event ordering



# PERFORMANCE OF COMMUNICATION CHANNELS

- Communication over a computer network has the following performance characteristics relating to latency, bandwidth and jitter:
- The delay between the start of a message's transmission from one process and the beginning of its receipt by another is referred to as ***latency.***
  - The delay in accessing the network, which increases significantly when the network is heavily loaded. For example, for Ethernet transmission the sending station waits for the network to be free of traffic.
  - The time taken by the operating system communication services at both the sending and the receiving processes, which varies according to the current load on the operating systems.
- The bandwidth of a computer network is the total amount of information that can be transmitted over it in a given time. When a large number of communication channels are using the same network, they have to share the available bandwidth.

- ***Jitter*** is the variation in the time taken to deliver a series of messages. Jitter is relevant to multimedia data. For example, if consecutive samples of audio data are played with differing time intervals, the sound will be badly distorted.

## Computer clocks and timing events

- Each computer in a distributed system has its own internal clock, which can be used by local processes to obtain the value of the current time. Therefore two processes running on different computers can each associate timestamps with their events. However, even if the two processes read their clocks at the same time, their local clocks may supply different time values.
- There are several approaches to correcting the times on computer clocks.
- The resulting agreement between the times on the local clocks is, of course, affected by variable message delays.

- Two variants of the interaction model
- In a distributed system it is hard to set limits on the time that can be taken for process execution, message delivery or clock drift.
- Two opposing extreme positions provide a pair of simple models – the first has a strong assumption of time and the second makes no assumptions about time:
- ***Synchronous distributed systems***: Hadzilacos and Toueg [1994] define synchronous distributed system to be one in which the following bounds are defined:

- The time to execute each step of a process has known lower and upper bounds.
- Each message transmitted over a channel is received within a known bounded time.
- Each process has a local clock whose drift rate from real time has a known bound.

- ***Asynchronous distributed systems***: Many distributed systems, such as the Internet, are very useful without being able to qualify as synchronous systems. Therefore we need an alternative model. An asynchronous distributed system is one in which there are no bounds on:
- Process execution speeds – for example, one process step may take only a picosecond and another a century; all that can be said is that each step may take an arbitrarily long time.
- Message transmission delays – for example, one message from process A to process B may be delivered in negligible time and another may take several years. In other words, a message may be received after an arbitrarily long time.
- Clock drift rates – again, the drift rate of a clock is arbitrary.

# TWO VARIANTS OF THE INTERACTION MODEL

*

- Synchronous distributed system
  - The time to execute each step f a process has known lower and uper bounds.
  - Each message transmitted over a channel is received within a known bounded time
  - Each process has a local clock whose drift rate from real time has a known bound.
- Asynchronous distributed system
  - No bound on process executiong speeds
  - No bound on message transmisson delays
  - No bound on clock drift rates.



# EVENT ORDERING

- In many cases, we are interested in knowing whether an event (sending or receiving a message) at one process occurred before, after or concurrently with another event at another process. The execution of a system can be described in terms of events and their ordering despite the lack of accurate clocks.
- For example, consider the following set of exchanges between a group of email users, X, Y, Z and A, on a mailing list:

1. User X sends a message with the subject *Meeting*.
2. Users Y and Z reply by sending a message with the subject *Re: Meeting*.

In real time, X's message is sent first, and Y reads it and replies; Z then reads both X's message and Y's reply and sends another reply, which references both X's and Y's messages. But due to the independent delays in message delivery, the messages may be delivered as shown in Figure 2.13, and some users may view these two messages in the wrong order. For example, user A might see:

| *Inbox:* | | |
|---|---|---|
| *Item* | *From* | *Subject* |
| 23 | Z | Re: Meeting |
| 24 | X | Meeting |
| 25 | Y | Re: Meeting |

If the clocks on X's, Y's and Z's computers could be synchronized, then each message could carry the time on the local computer's clock when it was sent. For example, messages $m_1$, $m_2$ and $m_3$ would carry times $t_1$, $t_2$ and $t_3$ where $t_1<t_2<t_3$. The messages received will be displayed to users according to their time ordering. If the clocks are roughly synchronized, then these timestamps will often be in the correct order.

Since clocks cannot be synchronized perfectly across a distributed system, Lamport [1978] proposed a model of *logical time* that can be used to provide an ordering among the events at processes running in different computers in a distributed system.

# FAILURE MODEL

- Omission failures
- Arbitrary failures
- Failure detection
- Impossibility of reaching agreement in the presence of failure
- Masking failure
- Reliability of one to one communication



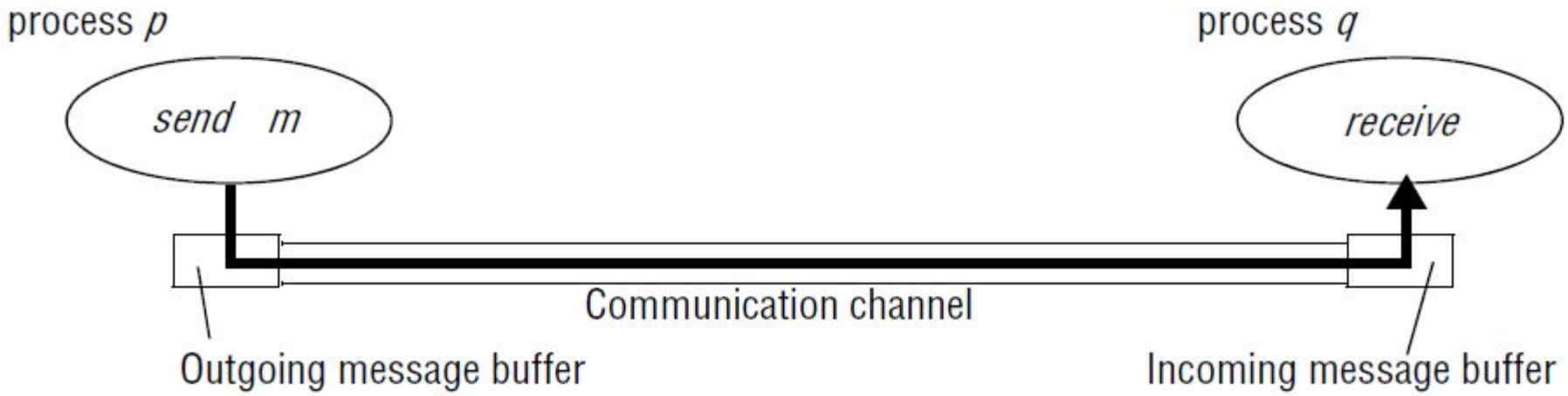
- Omission failures

The faults classified as *omission failures* refer to cases when a **process** or **communication channel fails .**

Process omission failures: The chief omission failure of a process is to crash. When we say that a process has crashed we mean that it has halted and will not execute any further steps of its program ever.

- A process crash is called fail-stop if other processes can detect certainly that the process has crashed. Fail-stop behavior can be produced in a synchronous system if the processes use timeouts to detect when other processes fail to respond and messages are guaranteed to be delivered.
- For example, if processes p and q are programmed for q to reply to a message from p, and if process p has received no reply from process q in a maximum time measured on p's local clock, then process p may conclude that process q has failed.

- Communication omission failures:
- Consider the communication primitives *send* and *receive*. A process *p* performs a *send* by inserting the message *m* in its outgoing message buffer. The communication channel transports *m* to *q*'s incoming message buffer.
- Process *q* performs a *receive* by taking *m* from its incoming message buffer and delivering it




The communication channel produces an omission failure if it does not transport a message from *p*'s outgoing message buffer to *q*'s incoming message buffer. This is known as 'dropping messages' and is generally caused by lack of buffer space

- Hadzilacos and Toueg [1994] refer to the loss of messages between the sending process and the outgoing message buffer as ***sendomission failures***, to loss of messages between the incoming message buffer and the receiving process as ***receive-omission failures***, and to loss of messages in between as *channel omission failures*.

Omission and arbitrary failures

| *Class of failure* | *Affects* | *Description* |
|---|---|---|
| Fail-stop | Process | Process halts and remains halted. Other processes may detect this state. |
| Crash | Process | Process halts and remains halted. Other processes may not be able to detect this state. |
| Omission | Channel | A message inserted in an outgoing message buffer never arrives at the other end's incoming message buffer. |
| Send-omission | Process | A process completes a *send* operation but the message is not put in its outgoing message buffer. |
| Receive-omission | Process | A message is put in a process's incoming message buffer, but that process does not receive it. |
| Arbitrary (Byzantine) | Process or channel | Process/channel exhibits arbitrary behaviour: it may send/transmit arbitrary messages at arbitrary times or commit omissions; a process may stop or take an incorrect step. |

- Arbitrary failures
- The term *arbitrary* or *Byzantine* failure is used to describe the worst possible failure semantics, in which any type of error may occur.
  - A process may set wrong values in its data items, or it may return a wrong value in response to an invocation.
- Timing failures •
  - Timing failures are applicable in synchronous distributed systems where time limits are set on process execution time, message delivery time and clock drift rate.
- Masking failure
  - Each component in a distributed system is generally constructed from a collection of other components. It is possible to construct reliable services from components that exhibit failures. For example, multiple servers that hold replicas of data can continue to provide a service when one of them crashes



- Reliability of one to one communication
  - *Validity: Any message in the outgoing message buffer is eventually delivered to the* incoming message buffer.
  - *Integrity: The message received is identical to one sent, and no messages are* delivered twice.

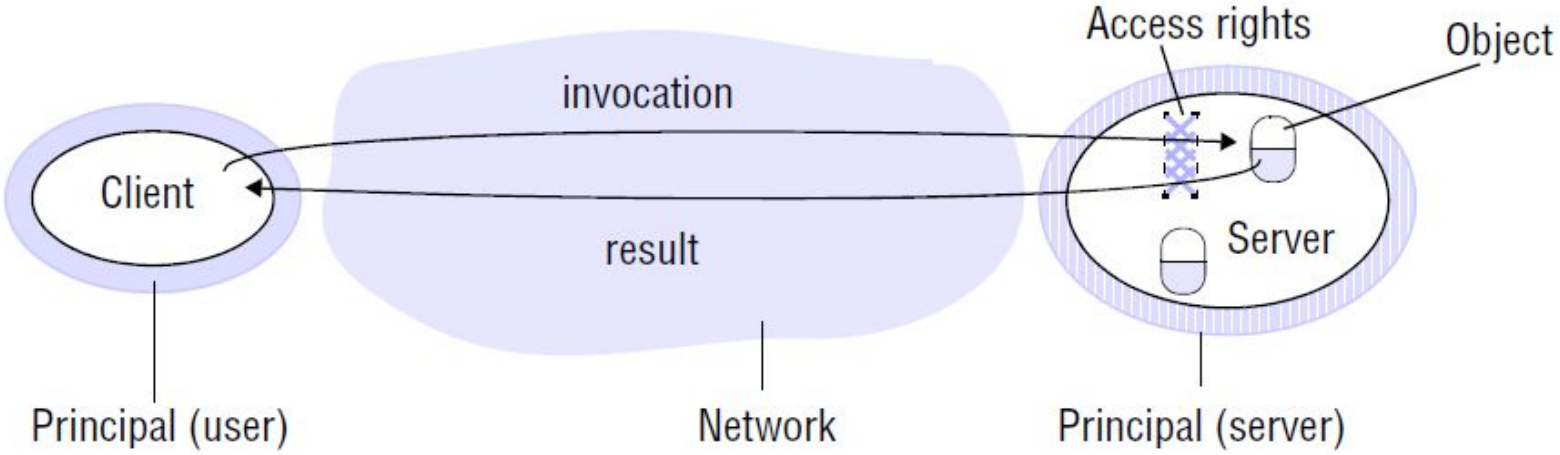


# SECURITY MODEL

- Protecting objects
- Securing processes and their interactions
- The enemy
- Defeating security threats
- Other possible threats from the enemy



Objects and principals



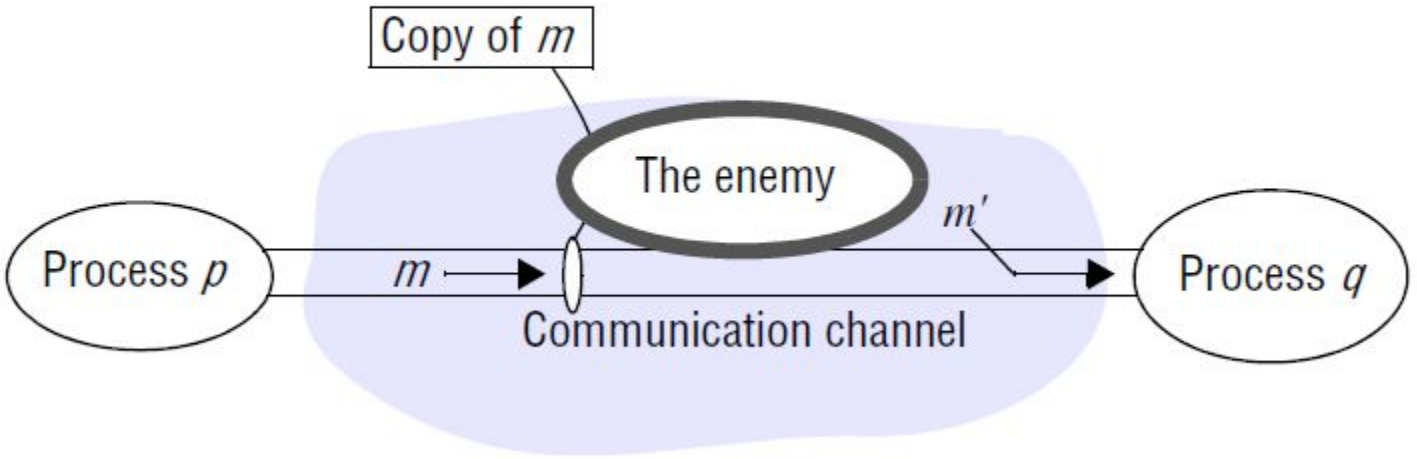

Objects are intended to be used in different ways by different users. For example, some objects may hold a user's private data, such as their mailbox, and other objects may hold shared data such as web pages. To support this, *access rights* specify who is allowed to perform the operations of an object – for example, who is allowed to read or to write its state.

## Securing processes and their interactions

The server is responsible for verifying the identity of the principal behind each invocation and checking that they have sufficient access rights to perform the requested operation on the particular object invoked, rejecting those that do not.

The client may check the identity of the principal behind the server to ensure that the result comes from the required server.

The enemy



## Defeating security threats

### Cryptography and shared secrets

### Authentication:

### Secure channels:

A secure channel ensures the privacy and integrity (protection against tampering) of the data transmitted across it.

- Each message includes a physical or logical timestamp to prevent messages from being replayed or reordered.

## Other possible threats from an enemy

*Denial of service*:

*Mobile code*

# EXERCISE

- The host computers used in peer-to-peer systems are often simply desktop computers in users' offices or homes. What are the implications of this for the availability and security of any shared data objects that they hold and to what extent can any weaknesses be overcome through the use of replication? *pages 47, 48*
- List the types of local resource that are vulnerable to an attack by an untrusted program that is downloaded from a remote site and run in a local computer *page 50*

# 5. SUMMARY

- Definitions of distributed systems and comparisons to centralized systems.
- The characteristics of distributed systems.
- The eight forms of transparency.
- The basic design issues.
- Read Chapter 1 and Chapter 2 of the textbook.

(DISTRIBUTED SYSTEMS Concepts and Design Fifth Edition George Coulouris)